

The Block-STM scheduler is the piece that turns “run txs in parallel, then fix conflicts” into a correct, lock-light protocol. MVCC stores versions. The scheduler decides **who runs, who validates, who aborts, and when the block is done** — without a giant priority queue.

The constraint that forces this design

Consensus already fixed an order $T_0 \prec T_1 \prec \dots \prec T_{n-1}$. Commit must match that serial order.

So:

- A successful validate of T_k is **not** a commit. T_j for $j < k$ can still abort and rewrite keys T_k read.
- You must keep re-validating higher txs until every prefix is stable.
- The same incarnation must never execute on two threads.
- Lower indices matter more: fixing T_2 unblocks the rest of the block.

A static “thread i owns txn i ” mapping fails. You need a **collaborative** pool that always prefers the lowest pending work.

Two kinds of work

Abstractly the scheduler is two ordered sets (no duplicates):

Set	Meaning	Starts as
E	incarnations waiting to execute	all txs, incarnation 0
V	incarnations waiting to validate	empty

Workers always take the **smallest index** in $E \cup V$.

Execution task. Run VM incarnation i of T_k against MVMemory.

- Read hits ESTIMATE → stop, put T_k back in E (dependency).
- Otherwise publish the write set.
- If this incarnation wrote a **new key** the previous incarnation did not, every $T_{\geq k}$ that is not already executing must be put in V (their reads may have changed).

Validation task. Re-read T_k ’s recorded read set. Compare versions.

- Pass → do nothing to state (still not committed).
- Fail → `try_abort` : mark write set `ESTIMATE` , bump incarnation, put T_k in E , put all $T_{>k}$ not executing into V .

Only the **first** abort of a given $(txn, incarnation)$ wins; later validate threads racing on the same version no-op.

Why not two priority queues

Concurrent heaps do not scale. Block-STM exploits the fact that “priority” is the txn index, a dense integer in $0..n$.

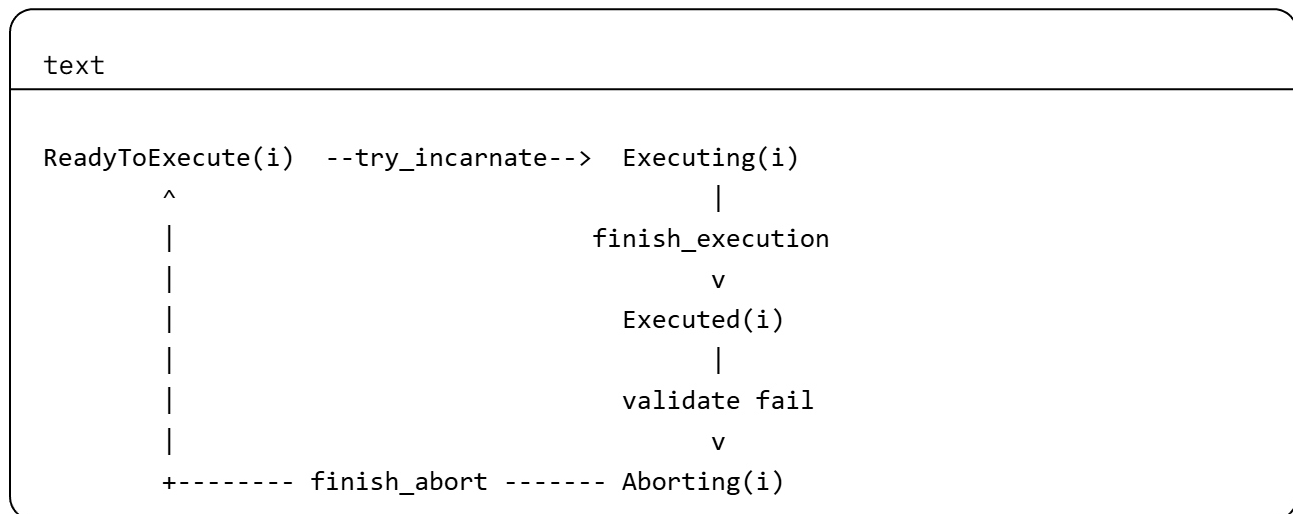
Replace each ordered set with:

- an atomic cursor (`execution_idx` / `validation_idx`)
- a per-txn status word

“Take the min ready element” becomes: `fetch_add` the cursor, look at that txn’s status, skip if not ready. When new work appears at a **lower** index, `fetch_min` the cursor back down. That is the whole concurrent set.

Per-transaction status

Classic V1 machine (paper / early Aptos):



Production Aptos also has `Suspended(i)` when execution hits a dependency (`ESTIMATE` / unresolved read) and `ExecutionHalted` if the block is torn down. `try_incarnate` is a mutex on that one status slot: only one thread can move `Ready` → `Executing` for

incarnation i . Incarnation numbers never decrease, so a version is incarnated at most once.

The counters

text

execution_idx	lower bound on txs that may still need execute
validation_idx	lower bound on txs that may still need validate
decrease_cnt	how often those cursors moved backward (for done-check)
num_active_tasks	in-flight execute/validate (RAII bump)
done_marker	block finished

next_version_to_execute :

1. `idx = execution_idx.fetch_add(1)`
2. if status is `Ready(i) → try_incarnate → return ExecutionTask(idx, i)`
3. else skip (another thread owns it, or not ready)

next_version_to_validate :

1. `idx = validation_idx.fetch_add(1)`
2. if status is `Executed(i) → return ValidationTask(idx, i, wave)`
3. else skip

`next_task` picks the smaller of the two cursors. If `validation_idx < execution_idx`, prefer validate. That keeps the prefix of the block “clean” so commit can advance.

When work is created **behind** a cursor:

text

<code>decrease_validation_idx(target)</code>	<code>→ validation_idx = min(validation_idx, target)</code>
<code>decrease_execution_idx(target)</code>	<code>→ execution_idx = min(execution_idx, target)</code>
<code>decrease_cnt++</code>	

Typical calls:

Event

Cursor move

T_k aborted	$\text{validation_idx} \leftarrow \min(., k+1), T_k \rightarrow \text{Ready, maybe}$ $\text{execution_idx} \leftarrow \min(., k)$
T_k wrote a new key	$\text{validation_idx} \leftarrow \min(., k)$ so T_k and everyone after re-validate
dependency resolved	$\text{execution_idx} \leftarrow \min(., \text{waiter})$

Without the decrease, workers would only walk forward and miss the newly dirty prefix.

Waves (Aptos production)

A raw `fetch_min` on `validation_idx` races with in-flight validators. Aptos packs **index + wave** in one `u64`: every time the cursor is pulled backward, `wave++`.

Each txn tracks:

- `max_triggered_wave` — last wave that required it to be validated
- `required_wave` — wave stamped onto the validation task
- `maybe_max_validated_wave` — highest wave that already passed

Commit of T_k is allowed only when its validated wave covers every trigger that could have invalidated it. That way a validate that started **before** an abort is not mistaken for a validate that ran **after** the abort.

Think of a wave as a generation number on “the world might have changed below you.”

Worker loop

text

```

loop:
  if done_marker: return
  task = scheduler.next_task()
  match task:
    Execution(k, inc):
      result = VM.execute(k, inc)          // reads MVMemory
      if NeedDependency(j):
        suspend k; when j finishes, Ready(k); decrease execution_idx
      else:
        publish writes
        scheduler.finish_execution(k, inc, wrote_new_keys?)
    Validation(k, inc, wave):
      ok = validate_read_set(k, inc)
      if !ok && scheduler.try_abort(k, inc):
        mark writes ESTIMATE
        scheduler.finish_abort(k, inc)    // Ready(inc+1), decrease idxs
  Retry:
    continue

```

`finish_execution` often returns `Retry` instead of a new task: the thread goes back to `next_task` so the global cursors stay fair. After abort, `finish_abort` may **hand the same thread** `ExecutionTask(k, inc+1)` immediately — re-execute the victim before wandering off to T_{900} . That is the “prioritize lower index” rule in code.

When is the block done?

Not “every txn validated once.” Done when:

```

\min(\textit{execution_idx}, \textit{validation_idx}) \geq n \;; \wedge \;
\textit{num_active_tasks} = 0 \;; \wedge \; \textit{decrease_cnt} \text{ is stable}

```

The last clause stops a race: a thread snapshots the cursors as “past the end” while another thread is about to `fetch_min` them backward. If `decrease_cnt` moved, you are not done. Then `done_marker = true` and workers exit. A separate **commit pointer** walks $0..n$ once statuses are `Executed` and waves line up, applying the highest MVMemory cell per key to storage.

Why this works (intuition)

1. MVMemory + READLAST $\Rightarrow T_k$ never sees writes from $T_{>k}$.

2. **Validate read set** $\Rightarrow$ if anything $T_{<k}$ published after T_k ran, T_k aborts.
3. **ESTIMATE + decrease validation_idx** $\Rightarrow$ everyone above finds out.
4. **try_incarnate** $\Rightarrow$ one owner per version.
5. **Atomic cursors** $\Rightarrow$ cheap “min heap” over $0..n$.
6. **Waves** $\Rightarrow$ stale in-flight validates cannot commit a dirty prefix.

Safety: final state = serial execution of the block. Liveness: each abort increments an incarnation; the paper proves no deadlock (waits only follow index order / ESTIMATE of a lower txn) and no livelock under fair scheduling.

V1 vs V2 (what you will see in aptos-core)

	Block-STM V1 (paper)	Aptos Block-STM V2
Validation	Explicit tasks in V , waves of <code>next_task</code>	Many validations are on-demand when a lower txn publishes
Modules	Ordinary keys	Separate cold validation worker for published Move modules (module cache cannot push-invalidate)
Suspend	Abort + retry from scratch on ESTIMATE	First-class Suspended + resume
Commit	After global done	Streaming commit of a clean prefix while the tail still executes

The counting scheduler is the same idea; V2 moves some validate work off the global queue so contended blocks do not drown in suffix validation tasks.

Mental picture

text

```
execution_idx -----> [Ready][Exec][Exec][Ready][Executed] ...  
validation_idx ----->
```

```
abort T2:
```

```
  writes(T2) := ESTIMATE
```

```
  T2.incarnation += 1
```

```
  T2.status = Ready
```

```
  validation_idx = min(validation_idx, 3)
```

```
  execution_idx  = min(execution_idx, 2)
```

```
workers flock back to T2, then re-validate T3..
```

The scheduler is not a thread-per-tx map and not a lock on the whole block. It is two atomic high-water marks plus a tiny status word per txn, engineered so that **the next thing that must be correct in serial order is the next thing a core picks up**